

An Analytical Framework for Multi-Variable Non-Linear Damage Superposition in Bounded Discrete Tactical Environments

BotAn

Brown Dust II Tactical Compendium

June 17, 2026

Abstract

Predicting discrete state-vector outputs in complex tactical computational environments remains an analytical challenge when underlying source code is obfuscated. This paper presents a phenomenological framework that maps the conditional mechanics of the Brown Dust II engine into a unified, continuous tensor equation. Rather than relying on fragmented imperative control-flow logic, a declarative model is constructed from bottom-up empirical constraints. By representing elemental attributes as orthogonal property vectors and mapping modifiers via multidimensional linear transformations, a mathematically rigorous alternative to traditional algorithmic scripting is provided. Empirical validation across live server environments demonstrates absolute convergence ($R^2 = 1.0$), proving that observational modeling can perfectly reconstruct hidden system rules.

1 Introduction

In discrete tactical simulation environments, the determination of state transitions—specifically, the calculation of kinetic and ethereal energy transfer between opposing nodes—is a critical computational process. Modern turn-based architectures have evolved past elementary linear scaling systems, introducing multi-layered, interdependent modifier pools. While these complex mechanics enrich the strategic depth of the environment, they frequently introduce severe optimization challenges for system analysis.

Within the specific framework of the Brown Dust II tactical engine, automated data routing and character configuration require a precise understanding of how discrete attributes scale under active status modifiers. In practical execution, players and developers alike encounter a high-dimensional combinatorial optimization problem when configuring team sequencing matrices. Because the underlying source code of the execution engine remains proprietary and obfuscated, the precise topological structure of these interactions cannot be verified through direct code inspection.

This systemic lack of transparency presents a distinct analytical hurdle. When multiple status-altering modifiers—such as offensive amplifications (e.g., Attack or Magic Attack pooling), temporal damage vectors

(Chains), and structural vulnerabilities—interact simultaneously, predicting the exact output state becomes non-trivial. Compounding this challenge is the presence of hard operational boundary limits, such as saturation caps applied to entity vitality metrics, which inject non-linear constraints into otherwise predictable algebraic equations.

To resolve these ambiguities, this study adopts a black-box phenomenological approach to reverse-engineer and formalize the global damage system. By conducting systematic, isolated live-server experiments, individual systemic rules are identified, evaluated, and translated from sequential control-flow constraints into a unified mathematical syntax. It is demonstrated that apparent algorithmic discrepancies—such as the transition between additive pooling and multiplicative compounding—can be elegantly modeled using linear algebra and orthogonal basis vector operations.

The primary contributions of this paper are threefold:

1. A modular, bottom-up mathematical derivation of individual modifier behaviors constructed entirely from empirical observation.
2. The formulation of a unified master tensor equation that bypasses the need for imperative if/else control logic.
3. Computational verification proving that the proposed declarative model achieves absolute predictive convergence with live system executions.

The structural composition of the remainder of this document is organized as follows. Section 2 details the phase-by-phase axiomatic construction of individual scaling components and establishes the mathematical rationale behind additive buff stacking constraints. Section ?? synthesizes these isolated components into a single, comprehensive master equation. Section ?? presents empirical validation scenarios derived from live server readouts to verify the predictive robustness of the model. Finally, Section ?? leverages the formalized framework to outline the absolute mathematical boundaries of team matrix optimization.

2 System Modeling and Core Vector Mechanics

To construct a verifiable mathematical model of the calculation pipeline, the game’s underlying execution engine is analyzed by gradually introducing systemic complexities. This progressive approach ensures that the mathematical framework remains tightly aligned with observable live-server behavior.

2.1 The Elementary Scalar Model

The most intuitive baseline model assumes a standard physical interaction where a source unit executes an action against a single target tile occupied by a neutral target unit. In this idealized state, devoid of active status modifiers, elemental properties, or defensive mitigation mechanics, the primary baseline damage output (D_0) can be expressed as a basic scalar product:

$$D_0 = \text{ATK} \times M \quad (1)$$

where ATK represents the flat physical attack attribute of the source unit, and M denotes the scaling multiplier dictated by the active skill. While Equation 1 successfully accounts for elementary, single-variable physical interactions, it quickly fails when confronted with more sophisticated unit skill sets.

2.2 The Attribute Dependency Conflict

Empirical testing across diverse unit skill sets reveals that the calculation engine does not utilize a single, uniform attribute for output evaluations. Instead, independent character and costume skill designs shift their scaling dependencies dynamically across entirely distinct statistical pools:

1. **Magical Payloads:** Certain skill sets completely swap physical parameters for a magical alternative, forcing the baseline calculation to rely on $\text{MATK} \times M$
2. **Vitality Scaling:** Advanced tank-type units leverage alternative skill designs that bypass offensive statistics entirely, generating an output value scaled by the source unit’s own maximum health points ($\text{HP} \times M$).
3. **Simultaneous Multi-Attribute Scaling:** The most critical analytical conflict occurs with hybrid skill sets that track multiple independent metrics simultaneously. For example, specific protective mechanics compute an execution value based on the concurrent summation of both the source unit’s current Health Points and their active protective barrier layers (Energy Guard, EG) on a localized target tile.

Attempting to model these fluid, multi-variable dependencies using standard scalar mathematics would require a disjointed set of nested conditional control statements.

To maintain a clean, declarative mathematical framework that handles single-attribute and multi-attribute calculations uniformly, a structural shift from scalar spaces to vector spaces is required.

2.3 Vector Space Formalization and Operational Constraints

To elegantly resolve the multi-attribute dependencies identified above, alongside interactive mechanics that scale dynamically based on target unit metrics (such as target-punishment or executioner scaling properties), the interaction state is compiled into a unified, seven-dimensional column vector, designated as the Source-Target Stat Vector, $\vec{v} \in \mathbb{R}^7$:

$$\vec{v} = \begin{pmatrix} \text{ATK}_{\text{src}} \\ \text{MATK}_{\text{src}} \\ \text{HP}_{\text{src}} \\ \text{EG}_{\text{src}} \\ \text{ATK}_{\text{tgt}} \\ \text{MATK}_{\text{tgt}} \\ \text{HP}_{\text{tgt}} \end{pmatrix} \quad (2)$$

This vertical coordinate space maps the primary offensive parameters alongside vitality pools for both the source entity and the target entity simultaneously.

At the database level, the system enforces strict structural boundaries on the elements of $\vec{v}$ based on the character templates assigned to the respective units. Because any individual unit asset is hardcoded to belong exclusively to either a physical or a magical archetype, the active offensive parameters within the vector space must satisfy a pair of decoupled zero-product boundary conditions:

$$v_1 \cdot v_2 = 0 \quad (\text{ATK}_{\text{src}} \cdot \text{MATK}_{\text{src}} = 0) \quad (3)$$

$$v_5 \cdot v_6 = 0 \quad (\text{ATK}_{\text{tgt}} \cdot \text{MATK}_{\text{tgt}} = 0) \quad (4)$$

These conditions ensure that if a unit configuration yields active physical parameters ($\text{ATK} \neq 0$), its corresponding magical component is fundamentally driven to zero ($\text{MATK} = 0$), and vice versa. Vitality metrics face no such structural constraints and exist simultaneously across all active unit entities.

To interface with this combined attribute snapshot, the executing skill set maps its specific scaling coefficients into a matching seven-dimensional column vector, designated as the Skill Scaling Vector, $\vec{m} \in \mathbb{R}^7$

$$\vec{m} = (m_1 \ m_2 \ m_3 \ m_4 \ m_5 \ m_6 \ m_7)^\top \quad (5)$$

All individual scaling coefficients within the Skill Scaling Vector are bound by a strict non-negativity constraint, such that $m_i \geq 0$ for all element index positions $i \in \{1, \dots, 7\}$. This structural floor ensures that a baseline skillset execution can only generate constructive or neutral payload components, mathematically isolating all reductive transformations to the external debuff pools and adversarial mitigation layers evaluated later in the pipeline.

To calculate the active interaction payload components across all seven statistical channels simultaneously,

the engine executes an element-wise Hadamard product ($\circ$) between the two column vectors, creating an intermediate column Execution Vector, $\vec{e} \in \mathbb{R}^7$:

$$\vec{e} = \vec{v} \circ \vec{m} \quad (6)$$

where each individual element is evaluated via scalar multiplication as $e_i = v_i \cdot m_i$. Utilizing the element-wise mapping of the Hadamard product is mathematically mandatory for this system configuration; it ensures that independent statistical properties remain completely isolated before final summation. This architectural choice allows subsequent environmental modifier fields to be mapped onto the system as independent vector operations.

The final, unmitigated base damage output (D_0) for the targeted tile is then extracted by evaluating the sum of the resulting execution elements:

$$D_0 = \sum_{i=1}^7 e_i = \sum_{i=1}^7 v_i \cdot m_i \quad (7)$$

This formulation successfully unifies the underlying system mechanics under a single declarative rule. For a standard physical interaction, Equations 3 and 4 force the magical elements of the active unit profiles to zero. Consequently, the intermediate Execution Vector $\vec{e}$ evaluates to an axis-aligned state containing only the active physical payload components, rendering the final summation in Equation 7 mathematically equivalent to an elementary scalar model. For hybrid or target-interactive skill sets, the Hadamard framework natively preserves the distinct parallel channels for vitality metrics and target scaling values, mapping out their simultaneous contributions before final aggregation without requiring fragmented conditional code branches.

2.4 Base Attribute Construction and Level Dependencies

The individual elements compiled within the Source-Target Stat Vector $\vec{v}$ are not primitive, static constants pulled directly from character sheets. Instead, a structural division exists between persistent, macro-progression attributes controlled by the source entity, transient combat barrier metrics, and external values mirrored from the target state node.

For the native, persistent source attribute channels v_i where $i \in \{1, 2, 3\}$ (corresponding explicitly to ATK_{src} , MATK_{src} , and HP_{src} respectively), each element represents the dynamic terminal output of a multi-layered Attribute Aggregation Phase. Live-server empirical research reveals that the engine evaluates a strict hierarchical sequence of level-scaling, flat additions, and global percentage modifiers to construct these final pre-combat stat parameters before grid deployment.

To formalize this persistent pipeline, let S_{base} represent the initial, native baseline value of a given source attribute channel i . The level-dependent growth curve is driven by a linear scaling coefficient, α_i , determined

explicitly by the category of the target attribute:

$$\alpha_i = \begin{cases} 0.1 & \text{if } i = 1 \text{ or } 2 \quad (\text{ATK}_{\text{src}} \text{ or } \text{MATK}_{\text{src}}) \\ 0.2 & \text{if } i = 3 \quad (\text{HP}_{\text{src}}) \end{cases} \quad (8)$$

Crucially, while Equation 8 evaluates a non-zero linear progression rate for both offensive channels, the structural boundary constraint established in Equation 3 dictates that the vacant archetype channel initiates with a baseline property value of $S_{\text{base}} = 0$, rendering its concurrent level aggregation trivially zero.

By utilizing this coefficient alongside the unit's current level (L), the absolute flat component accumulation value ($v_{i,\text{flat}}$) aggregates native growth concurrently with flat numerical additions from gear configurations, active or permanent bond systems, and awakening milestones:

$$v_{i,\text{flat}} = (S_{\text{base}} + \alpha_i \cdot L \cdot S_{\text{base}}) + A_{\text{gear_flat}} + A_{\text{bond_flat}} + A_{\text{awake_flat}} \quad (9)$$

Once $v_{i,\text{flat}}$ is established, the engine evaluates a unified multiplicative layer containing all passive, permanent, and equipment-based percentage scaling pools to yield the compound coefficient channel, $v_{i,\text{field}}$:

$$v_{i,\text{field}} = 1 + \omega_{\text{collection}} + \omega_{\text{gear}} + \omega_{\text{awakening}} + \omega_{\text{bond}} \quad (10)$$

where each ω term represents the fractional value of its respective percentage bonus pool. The terminal active value loaded into these persistent vector coordinates is subsequently computed as the direct product of these two distinct structural phases:

$$v_i = v_{i,\text{flat}} \times v_{i,\text{field}} \quad \text{for } i \in \{1, 2, 3\} \quad (11)$$

In contrast to the persistent attribute pipeline, the protective barrier layer channel v_4 (EG_{src}) represents a highly volatile, dynamic instance state rather than an internal progression attribute. Because defensive shield parameters are generated at runtime through diverse external skill variables and non-native scaling references, v_4 fundamentally bypasses Equations 8 through 11. Instead, the engine isolates this slot from the macro-progression accumulation layer, populating v_4 directly via a real-time snapshot of the absolute active barrier shielding remaining on the source node's grid matrix location at the exact execution frame.

This synthesis successfully concludes the initialization and consolidation of the active attribute parameters. By fully encapsulating all character growth, equipment additions, and passive account progressions directly within the core source coordinates v_1 through v_4 , the system establishes a stable, fully aggregated dataset ready to be processed through the localized skillset execution layers, while elements v_5 through v_7 are concurrently mirrored from the corresponding target unit's independent aggregation layer at the moment of runtime execution.

2.5 Positive Constraint Enforcement and Rounding Behavior

To ensure that the final output damage value (D) obeys the game logical constraints, a strict positive constraint

is applied, such as $D = \max(D_0, 1)$. This ensures that the minimum damage output is always equal or greater than 1, preventing zero or negative damage values from being generated. Additionally, the engine applies a rounding function to the final damage output, which rounds down to the nearest integer. This is mathematically represented as $D = \lfloor D \rfloor$, ensuring that all damage outputs are whole numbers, consistent with the discrete nature of the tactical environment.

Combining these constraints, the final damage output at this stage can be expressed as:

$$D = \max(\lfloor D_0 \rfloor, 1) \quad (12)$$

2.6 Skill Scaling Vector Construction

2.6.1 Skills

Skill scaling parameters are derived from the skill's inherent design and are influenced by various factors such as Costume upgrade level, so-called Potential Liberation and Burst mechanics, as well as having conditional scaling dependencies on the target or source unit's current state. Generally speaking, each Skill Scaling Vector Element m_i can be expressed as follows:

$$m_i = m_i^{(\text{costume})} + \sum_j \Delta m_{i,j}^{(\text{liberation})} + \sum_j \Delta m_{i,j}^{(\text{burst})} \quad (13)$$

Note that $m_i^{(\text{costume})}$ is a function of the Costume Upgrade Level. These functions, as well as Elements for Potential Liberation and Burst mechanics, are displayed in appendix ??.

Both Costume and Potential Liberation Vector elements can contain conditional dependencies on the target or source unit's current state and will be also discussed in appendix ??.

2.6.2 Basic Attacks

Basic Attacks can be represented as a special case of the general skill scaling framework, where the scaling coefficients are determined by the character's base attack pattern and are not influenced by external factors such as costume upgrades or potential liberations. For Basic Attacks, the Skill Scaling Vector Elements m_1 and m_2 (corresponding to physical and magical attack scaling respectively) can be both set to 1 due to constraint on the Source-Target Stat Vector, while all other elements are set to 0, resulting in the following Skill Scaling Vector:

$$\vec{m}_{\text{BA}} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (14)$$

2.6.3 Knockbacks

Knockbacks is another special case of the general skill scaling framework, where the damage is always set to

1 (do not confuse with collision damage caused by the knockback itself). Since the damage output is fixed, the Skill Scaling Vector for Knockbacks can be represented as a null vector:

$$\vec{m}_{\text{KB}} = \vec{0} \quad (15)$$

In other words, all elements of the Skill Scaling Vector for Knockbacks are set to 0, indicating that the damage output does not scale with any of the source or target attributes.

2.6.4 Collision Damage

Since the behavior we are modeling is the direct source-target interaction, collision damage is not considered in this framework. However, we can model collision damage as secondary interaction between knocked back unit (source) and the unit that is being collided into (target), with the m_3 that represents the scaling coefficient for the source unit's HP, changing based on the initial source unit's parameters.

For the collision damage inherited from the simple knockback, m_3 is set to a fixed value of 0.25, while all other elements are set to 0, resulting in the following Skill Scaling Vector:

$$\vec{m}_{\text{CD}} = \begin{pmatrix} 0 \\ 0 \\ 0.25 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (16)$$

However, if the knockback is caused by a skill that has different scaling coefficient for the knocked back unit's HP, then m_3 will be set to that scaling coefficient instead of 0.25.

A Complete Skill Multiplier Registry

This appendix contains the comprehensive, state-dependent algebraic expressions and progression parameters for all character costume skills evaluated within this study.

For compact representation within Table 1, when an indexed range of scaling parameters is set equal to a scalar zero (e.g., $M_{2...7} = 0$ or $\Delta m_{i,2} = 0$), it denotes that the scalar valuation of 0 applies uniformly across all coordinate elements within that specified index boundary for all costume duplicate tiers c .

- δ_{MT} : Main Target Indicator. Evaluates to 1 if the calculation is evaluating the targeted node (the direct impact cell/primary target), and 0 for secondary nodes caught within the skill area-of-effect grid geometry.
- δ_{DoT} : Damage-over-Time Indicator. Evaluates to 1 if the calculation is evaluating a damage-over-time tick, and 0 for the initial hit instance.
- N_{DoT} : Represents the current number of concurrent status effect stacks applied to the target entity, where $N_{\text{DoT}} \in \{1, \dots, \Omega\}$ and Ω represents the maximum stack boundary dictated by the specific skill mechanics (e.g., $\Omega = 3$ for Angelica, $\Omega = 99$ for Dalvi).
- δ_{Counter} : Counter Indicator. Evaluates to 1 if the calculation is evaluating a counter damage, and 0 for the initial hit instance.

Table 1: Comprehensive Character Costume Skill Scaling Registry

Character (Costume)	$M_i[c]$	$\Delta m_{i,j}^{(\text{liberation})}$	$\Delta m_{i,j}^{(\text{burst})}$
Alec (Sword Breaker)	$M_1 = [1.0, 1.4, 1.75, 1.75, 2.1, 2.4]$	$\Delta m_{1,1} = 0.6$	$\Delta m_{i,j} = 0$
	$M_{2,...,7} = 0$	$\Delta m_{2...7,1} = \Delta m_{i,2} = \Delta m_{i,3} = 0$	
Alec (The Destruction)	$M_1 = [4.0, 5.25, 6.25, 6.25, 7.25, 8.0]$	$\Delta m_{1,1} = \Delta m_{1,2} = 2.0$	$\Delta m_{i,j} = 0$
	$M_{2,...,7} = 0$	$\Delta m_{2...7,1} = \Delta m_{2...7,2} = \Delta m_{i,3} = 0$	
Anastasia (Fire Graffiti)	$M_1 = [0.3 + 0.25 \cdot \delta_{\text{MT}}$	$\Delta m_{1,1} = \Delta m_{1,2} = 0.05$ $\Delta m_{2...7,1} = \Delta m_{2...7,2} = \Delta m_{i,3} = 0$	$\Delta m_{1,1} = \Delta m_{1,2} = 0.3$ $\Delta m_{2...7,1...2} = \Delta m_{i,3} = 0$
	$0.3 + 0.3 \cdot \delta_{\text{MT}},$		
	$0.35 + 0.35 \cdot \delta_{\text{MT}},$		
	$0.35 + 0.35 \cdot \delta_{\text{MT}},$		
	$0.4 + 0.4 \cdot \delta_{\text{MT}},$		
	$0.4 + 0.4 \cdot \delta_{\text{MT}}]$		
	$M_{2,...,7} = 0$		

Continued on next page

Table 1: Comprehensive Character Costume Skill Scaling Registry

Character (Costume)	$M_i[c]$	$\Delta m_{i,j}^{(\text{liberation})}$	$\Delta m_{i,j}^{(\text{burst})}$
Anastasia (Gentle Maid)	$M_1 = [1.1 + 1.4 \cdot \delta_{\text{MT}},$ $1.1 + 1.9 \cdot \delta_{\text{MT}},$ $1.6 + 1.9 \cdot \delta_{\text{MT}},$ $1.6 + 1.9 \cdot \delta_{\text{MT}},$ $2.1 + 1.9 \cdot \delta_{\text{MT}},$ $2.1 + 1.9 \cdot \delta_{\text{MT}}]$ $M_{2,\dots,7} = 0$	$\Delta m_{1,1} = \Delta m_{1,2} = 1.0$ $\Delta m_{2\dots 7,1} = \Delta m_{i,2} = \Delta m_{i,3} = 0$	$\Delta m_{i,j} = 0$
Andrew (Loyal Butler)	$M_1 = [0.75, 0.75, 1.0, 1.0, 1.5, 2.25]$ $M_{2,\dots,7} = 0$	$\Delta m_{1,1} = 0.5$ $\Delta m_{2\dots 7,1} = \Delta m_{i,2} = \Delta m_{i,3} = 0$	$\Delta m_{i,j} = 0$
Andrew (Specialist)	$M_i = 0$	$\Delta m_{i,j} = 0$	$\Delta m_{i,j} = 0$
Angelica (Neon Savior)	$M_7 = [0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.14 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}},$ $0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.18 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}},$ $0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.21 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}},$ $0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.21 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}},$ $0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.23 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}},$ $0.05 \cdot (1 - \delta_{\text{DoT}}) + 0.25 \cdot \delta_{\text{DoT}} \cdot N_{\text{DoT}}]$ $M_{1,\dots,6} = 0$	$\Delta m_{7,2} = 0.12$ $\Delta m_{1\dots 6,2} = \Delta m_{i,1} = \Delta m_{i,3} = 0$	$\Delta m_{i,j} = 0$
Angelica (Pool Party)	$M_7 = [0.15, 0.18, 0.21, 0.21, 0.23, 0.25]$ $M_{1,\dots,6} = 0$	$\Delta m_{7,1} = \Delta m_{7,3} = 0.02$ $\Delta m_{1\dots 6,1} = \Delta m_{i,2} = \Delta m_{1\dots 6,3} = 0$	$\Delta m_{i,j} = 0$
Angelica (The Fallen)	$M_7 = [0.45, 0.55, 0.64, 0.64, 0.72, 0.8]$ $M_{1,\dots,6} = 0$	$\Delta m_{7,2\dots 3} = 0.05$ $\Delta m_{i,1} = \Delta m_{1\dots 6,2} = \Delta m_{1\dots 6,3} = 0$	$\Delta m_{i,j} = 0$
Arines (Priest of Vitality)	$M_i = 0$	$\Delta m_{i,j} = 0$	$\Delta m_{i,j} = 0$
Beatrice (The Mighty Warrior of the Tribe)	$M_1 = [1.25, 1.25, 1.5, 1.5, 1.75, 2.25]$ $M_{2,\dots,7} = 0$	$\Delta m_{1,1} = 0.75$ $\Delta m_{2\dots 7,1} = \Delta m_{i,2} = \Delta m_{i,3} = 0$	$\Delta m_{i,j} = 0$
Bernie (Righteous Raider Girl)	$M_1 = [0.5, 0.5, 0.7, 0.7, 1.1, 1.1]$ $M_{2,\dots,7} = 0$	$\Delta m_{i,j} = 0$	$\Delta m_{i,j} = 0$

Continued on next page

Table 1: Comprehensive Character Costume Skill Scaling Registry

Character (Costume)	$M_i[c]$	$\Delta m_{i,j}^{(\text{liberation})}$	$\Delta m_{i,j}^{(\text{burst})}$
Blade (Apostle)	$M_1 = [3.0 \cdot (1 - \delta_{\text{Counter}}) + 1.0 \cdot \delta_{\text{Counter}},$		
	$3.8 \cdot (1 - \delta_{\text{Counter}}) + 1.2 \cdot \delta_{\text{Counter}},$		
	$3.8 \cdot (1 - \delta_{\text{Counter}}) + 1.2 \cdot \delta_{\text{Counter}},$	$\Delta m_{1,1} = 0.8 \cdot (1 - \delta_{\text{Counter}})$	
	$4.6 \cdot (1 - \delta_{\text{Counter}}) + 1.4 \cdot \delta_{\text{Counter}},$	$\Delta m_{1,2} = 0.2 \cdot \delta_{\text{Counter}}$	$\Delta m_{i,j} = 0$
	$5.4 \cdot (1 - \delta_{\text{Counter}}) + 1.6 \cdot \delta_{\text{Counter}},$	$\Delta m_{2\dots7,1} = \Delta m_{2\dots7,2} = \Delta m_{i,3} = 0$	
	$6.2 \cdot (1 - \delta_{\text{Counter}}) + 1.8 \cdot \delta_{\text{Counter}}]$		
Blade (Onsen Swordfighter)	$M_{2,\dots,7} = 0$		
	$M_1 = [3.5 + 0.7 \cdot N_{\text{Debuff}},$		
	$3.5 + 0.7 \cdot N_{\text{Debuff}},$		
	$4.25 + 0.7 \cdot N_{\text{Debuff}},$	$\Delta m_{1,1} = \Delta m_{1,2} = 0.5$	
	$4.25 + 0.9 \cdot N_{\text{Debuff}},$	$\Delta m_{1,3} = 0.1 \cdot N_{\text{Debuff}}$	$\Delta m_{i,j} = 0$
	$5.0 + 0.9 \cdot N_{\text{Debuff}},$	$\Delta m_{2\dots7,1} = \Delta m_{2\dots7,2} = \Delta m_{2\dots7,3} = 0$	
Blade (Young Lady)	$5.0 + 1.1 \cdot N_{\text{Debuff}}]$		
	$M_{2,\dots,7} = 0$		
	$M_1 = [0.5, 0.5, 0.7, 0.7, 1.1, 1.3]$	$\Delta m_{1,1} = 0.2$	$\Delta m_{i,j} = 0$
	$M_{2,\dots,7} = 0$	$\Delta m_{2\dots7,1} = \Delta m_{i,2} = \Delta m_{i,3} = 0$	